CS 498 - Stage 3 Code Listing

MongoDB + InsideAirbnb prototype

Abhinav Bachu, Hanshul Bahl, Alejandra Arias

Generated 2026-04-24

Repository: `github.com/abachu2005/cs498-airbnb-mongodb-prototype`

Files included

- `package.json` — npm scripts and dependency manifest
- `src/loader.js` — streams insideairbnb csvs into mongo, builds eight stage-2 indexes
- `src/q1.js` — Q1 - highest-rated listings open for two consecutive nights
- `src/q2.js` — Q2 - neighborhoods with zero active listings in a given month
- `src/q3.js` — Q3 - per-listing bookable intervals, min-nights aware
- `src/q4.js` — Q4 - portland mar-aug bookable-nights trend (reuses Q3)
- `src/q5.js` — Q5 - december reviews per (city, year)
- `src/q6.js` — Q6 - re-book reminders + same-host listings
- `src/driver.js` — CLI runner: boots mongo, loads data, runs all 6 queries + explain
- `web/server.js` — express server exposing each query as an HTTP endpoint
- `web/public/index.html` — demo UI markup -- one tab per query
- `web/public/app.js` — demo UI client logic -- runQ/renderQ pair per query

package.json

npm scripts and dependency manifest

```
{
  "name": "stage3-prototype",
  "version": "1.0.0",
  "private": true,
  "type": "commonjs",
  "scripts": {
    "start": "node --max-old-space-size=8192 src/driver.js",
    "start:fast": "LIMIT_LISTINGS=500 LIMIT_REVIEWS=1000 LIMIT_NEIGHBORHOODS=200
LIMIT_CALENDAR=2000 node src/driver.js",
```

```
    "web": "node --max-old-space-size=8192 web/server.js",
    "web:fast": "LIMIT_LISTINGS=500 LIMIT_REVIEWS=1000 LIMIT_NEIGHBORHOODS=200
LIMIT_CALENDAR=2000 node web/server.js",
    "build:pdfs": "node scripts/build_pdfs.js",
    "build:codepdf": "node scripts/build_code_pdf.js"
  },
  "dependencies": {
    "csv-parse": "^6.2.1",
    "express": "^5.2.1",
    "mongodb": "^7.1.1",
    "mongodb-memory-server": "^11.0.1"
  },
  "devDependencies": {
    "md-to-pdf": "^5.2.5"
  }
}
```

src/loader.js

streams insideairbnb csvs into mongo, builds eight stage-2 indexes

```
// loader: streams the four insideairbnb csvs per city straight into mongo
// nothing here is held fully in memory -- the calendar files alone are 23M+ rows
// p = streaming csv parser, ps = sync parser used only for the small uncompressed
nbhd file
const z = require("zlib");
const { Readable } = require("stream");
const { parse: p } = require("csv-parse");
const { parse: ps } = require("csv-parse/sync");

// city -> dataset urls, snapshot dates baked in per city (insideairbnb publishes
per snapshot)
const S = {
  los_angeles: {
    listings: "https://data.insideairbnb.com/united-states/ca/los-angeles/2025-12-
04/data/listings.csv.gz",
    reviews: "https://data.insideairbnb.com/united-states/ca/los-angeles/2025-12-
04/data/reviews.csv.gz",
    neighborhoods: "https://data.insideairbnb.com/united-states/ca/los-angeles/2025-
12-04/visualisations/neighbourhoods.csv",
    calendar: "https://data.insideairbnb.com/united-states/ca/los-angeles/2025-12-
04/data/calendar.csv.gz",
  },
  portland: {
    listings: "https://data.insideairbnb.com/united-states/or/portland/2025-12-
04/data/listings.csv.gz",
    reviews: "https://data.insideairbnb.com/united-states/or/portland/2025-12-
04/data/reviews.csv.gz",
    neighborhoods: "https://data.insideairbnb.com/united-states/or/portland/2025-12-
04/visualisations/neighbourhoods.csv",
    calendar: "https://data.insideairbnb.com/united-states/or/portland/2025-12-
04/data/calendar.csv.gz",
  },
  salem: {
    listings: "https://data.insideairbnb.com/united-states/or/salem-or/2025-12-
28/data/listings.csv.gz",
    reviews: "https://data.insideairbnb.com/united-states/or/salem-or/2025-12-
28/data/reviews.csv.gz",
    neighborhoods: "https://data.insideairbnb.com/united-states/or/salem-or/2025-12-
28/visualisations/neighbourhoods.csv",
    calendar: "https://data.insideairbnb.com/united-states/or/salem-or/2025-12-
28/data/calendar.csv.gz",
  },
  san_diego: {
```

```
    listings: "https://data.insideairbnb.com/united-states/ca/san-diego/2025-09-
25/data/listings.csv.gz",
    reviews: "https://data.insideairbnb.com/united-states/ca/san-diego/2025-09-
25/data/reviews.csv.gz",
    neighborhoods: "https://data.insideairbnb.com/united-states/ca/san-diego/2025-
09-25/visualisations/neighbourhoods.csv",
    calendar: "https://data.insideairbnb.com/united-states/ca/san-diego/2025-09-
25/data/calendar.csv.gz",
  },
};

// env override, accepts a number or "all" / "Infinity", falls back to d
function envN(k, d) {
  const v = process.env[k];
  if (v == null || v === "") return d;
  if (v === "all" || v === "ALL" || v === "Infinity") return Infinity;
  const n = Number(v);
  return Number.isFinite(n) ? n : d;
}

// per-collection row caps, default Infinity = full insideairbnb snapshot
// LIMIT_* env vars let the fast/demo modes cap at a few thousand rows for ~10s
boots
const L = {
  listings: envN("LIMIT_LISTINGS", Infinity),
  reviews: envN("LIMIT_REVIEWS", Infinity),
  neighborhoods: envN("LIMIT_NEIGHBORHOODS", Infinity),
  calendar: envN("LIMIT_CALENDAR", Infinity),
};

// fetch a csv into an array (g = gzipped), used by callers that want everything in
memory
// kept around for small files only; the big csvs go through fStream instead
async function f(u, n, g) {
  const r = await fetch(u);
  if (!r.ok) throw new Error(`HTTP ${r.status} ${u}`);
  if (g) {
    const a = Readable.fromWeb(r.body);
    const b = z.createGunzip();
    const c = a.pipe(b).pipe(p({ columns: true, skip_empty_lines: true,
relax_column_count: true, relax_quotes: true }));
    const d = [];
    for await (const e of c) {
      d.push(e);
      if (d.length >= n) { c.destroy(); break; }
    }
    b.destroy();
    a.destroy();
    return d;
  }
```

```
  const t = Buffer.from(await r.arrayBuffer()).toString("utf-8");
  const w = ps(t, { columns: true, skip_empty_lines: true, relax_column_count:
true, relax_quotes: true });
  return n === Infinity ? w : w.slice(0, n);
}

// streaming variant: fetch -> gunzip -> csv parse -> shape via fn -> push into sink
// stops early once n rows seen, never holds the full file in memory (this is what
makes
// loading 16M LA calendar rows possible inside mongodb-memory-server)
async function fStream(u, n, g, fn, sink) {
  const r = await fetch(u);
  if (!r.ok) throw new Error(`HTTP ${r.status} ${u}`);
  if (!g) {
    const t = Buffer.from(await r.arrayBuffer()).toString("utf-8");
    const w = ps(t, { columns: true, skip_empty_lines: true, relax_column_count:
true, relax_quotes: true });
    const a = n === Infinity ? w : w.slice(0, n);
    let c = 0;
    for (const r of a) {
      const v = fn(r);
      if (v) {
        sink.push(v);
        c++;
      }
    }
    await sink.flush();
    return c;
  }
  const a = Readable.fromWeb(r.body);
  const b = z.createGunzip();
  const c = a.pipe(b).pipe(p({ columns: true, skip_empty_lines: true,
relax_column_count: true, relax_quotes: true }));
  let i = 0;
  for await (const e of c) {
    if (i >= n) {
      c.destroy();
      break;
    }
    const v = fn(e);
    if (v) await sink.push(v);
    i++;
  }
  await sink.flush();
  b.destroy();
  a.destroy();
  return i;
}

// batched insertMany sink so the heap stays flat regardless of total row count
```

```
// ordered:false lets a single bad row not abort the whole batch (insideairbnb has
occasional dupes)
function makeSink(coll, batch) {
  const b = batch || 5000;
  let buf = [];
  let total = 0;
  return {
    async push(d) {
      buf.push(d);
      if (buf.length >= b) {
        const x = buf;
        buf = [];
        await coll.insertMany(x, { ordered: false });
        total += x.length;
      }
    },
    async flush() {
      if (buf.length) {
        await coll.insertMany(buf, { ordered: false });
        total += buf.length;
        buf = [];
      }
    },
    get total() {
      return total;
    },
  };
}

// numeric coerce, strips $ and , so "$1,250.00" parses cleanly, returns null on bad
input
function n(v) {
  if (v === undefined || v === null || v === "") return null;
  const x = Number(String(v).replace(/[$,]/g, ""));
  return Number.isFinite(x) ? x : null;
}

// listings doc shape, c = city tag, r = raw csv row
// description is trimmed to 200 chars -- no query needs the full text and it bloats
mem
// neighbourhood_cleansed is preferred over neighbourhood (insideairbnb's curated
value)
function dl(c, r) {
  return {
    city: c,
    listing_id: n(r.id),
    name: r.name || null,
    neighborhood: r.neighbourhood_cleansed || r.neighbourhood || null,
    room_type: r.room_type || null,
    property_type: r.property_type || null,
```

```
    accommodates: n(r.accommodates),
    price: n(r.price),
    minimum_nights: n(r.minimum_nights),
    maximum_nights: n(r.maximum_nights),
    host_id: n(r.host_id),
    host_name: r.host_name || null,
    review_scores_rating: n(r.review_scores_rating),
    listing_url: r.listing_url || null,
    description: r.description ? r.description.substring(0, 200) : null,
    amenities: r.amenities || null,
  };
}

// reviews doc shape, comments trimmed to 200 chars (q5/q6 only need date + ids)
function dr(c, r) {
  return {
    city: c,
    listing_id: n(r.listing_id),
    review_id: n(r.id),
    date: r.date || null,
    reviewer_id: n(r.reviewer_id),
    reviewer_name: r.reviewer_name || null,
    comments: r.comments ? r.comments.substring(0, 200) : null,
  };
}

// neighborhoods doc shape -- insideairbnb uses two slightly different column names
across cities
function dn(c, r) {
  return {
    city: c,
    neighborhood: r.neighbourhood || r.neighbourhood_group || null,
  };
}

// calendar doc shape, "t"/"f" -> bool so q1/q2/q3 can match on `available: true`
directly
function dc(c, r) {
  return {
    city: c,
    listing_id: n(r.listing_id),
    date: r.date || null,
    available: r.available === "t",
    price: n(r.price),
    adjusted_price: n(r.adjusted_price),
    minimum_nights: n(r.minimum_nights),
    maximum_nights: n(r.maximum_nights),
  };
}
```

```
// boots all four collections, walks each city, indexes after load (faster than
indexing-as-we-go)
// batch sizes (2k/5k/1k/10k) tuned per collection so each insertMany stays under
~10MB on the wire
async function load(db) {
  const lc = db.collection("listings");
  const rc = db.collection("reviews");
  const nc = db.collection("neighborhoods");
  const cc = db.collection("calendar");

  // dedup set for neighborhoods -- insideairbnb sometimes lists the same nbhd twice
in one csv
  // and we have a unique index on (city, neighborhood) so collisions would crash
insertMany
  const seen = new Set();

  let TL = 0;
  let TR = 0;
  let TN = 0;
  let TC = 0;

  for (const [k, v] of Object.entries(S)) {
    console.log(`[loader] ${k} listings`);
    const sL = makeSink(lc, 2000);
    TL += await fStream(v.listings, L.listings, true, (r) => dl(k, r), sL);

    console.log(`[loader] ${k} reviews`);
    const sR = makeSink(rc, 5000);
    TR += await fStream(v.reviews, L.reviews, true, (r) => dr(k, r), sR);

    console.log(`[loader] ${k} neighborhoods`);
    const sN = makeSink(nc, 1000);
    TN += await fStream(v.neighborhoods, L.neighborhoods, false, (r) => {
      const m = dn(k, r);
      if (!m.neighborhood) return null;
      const q = `${m.city}|${m.neighborhood}`;
      if (seen.has(q)) return null;
      seen.add(q);
      return m;
    }, sN);

    console.log(`[loader] ${k} calendar`);
    const sC = makeSink(cc, 10000);
    TC += await fStream(v.calendar, L.calendar, true, (r) => dc(k, r), sC);
  }

  // the eight stage 2 indexes -- creating after the load is ~2x faster than
maintaining per-insert
  console.log("[loader] indexing");
  await lc.createIndex({ city: 1, listing_id: 1 }, { unique: true });   // pk-style
```

```
listing lookup
  await lc.createIndex({ city: 1, neighborhood: 1, room_type: 1 });     // q1 / q2
/ q3 candidate filter
  await lc.createIndex({ host_id: 1, city: 1 });                         // q6
same-host lookup (see report critique on key order)
  await cc.createIndex({ city: 1, listing_id: 1, date: 1 });             // per-
listing calendar scan (q3, q4, q6)
  await cc.createIndex({ city: 1, date: 1, available: 1 });              // date-
window scan (q1, q2)
  await rc.createIndex({ city: 1, listing_id: 1, date: 1 });             // per-
listing reviews (q6)
  await rc.createIndex({ reviewer_id: 1, listing_id: 1, date: 1 });      // q6
repeat-reviewer aggregation prefix
  await nc.createIndex({ city: 1, neighborhood: 1 }, { unique: true });  // q2
anti-join target + dedup safety net

  const cl = await lc.estimatedDocumentCount();
  const cr = await rc.estimatedDocumentCount();
  const cn = await nc.estimatedDocumentCount();
  const cc2 = await cc.estimatedDocumentCount();

  return { listings: cl, reviews: cr, neighborhoods: cn, calendar: cc2 };
}

module.exports = { load };
```

src/q1.js

Q1 - highest-rated listings open for two consecutive nights

```
// q1: highest-rated listings in <city> whose calendar is open for every date in
o.dates
// driven from calendar (smaller per-day match against the date-window index)
instead of
// listings -> calendar lookups (which would scan all listings before filtering)
function pipe(o) {
  return [
    // hits the (city, date, available) index, returns one calendar row per
(listing, date) pair
    { $match: { city: o.city, date: { $in: o.dates }, available: true } },
    // collapse to one doc per listing with its open-day count
    { $group: { _id: "$listing_id", a: { $sum: 1 } } },
    // keep only listings open on every requested date (count must equal
dates.length)
    { $match: { a: o.dates.length } },
    // pull listing-level fields for display, joined via the unique (city,
listing_id) index
    {
      $lookup: {
        from: "listings",
        let: { l: "$_id" },
        pipeline: [
          { $match: { $expr: { $and: [{ $eq: ["$city", o.city] }, { $eq:
["$listing_id", "$$l"] }] } } },
          {
            $project: {
              _id: 0,
              listing_id: 1,
              name: 1,
              neighborhood: 1,
              room_type: 1,
              accommodates: 1,
              property_type: 1,
              amenities: 1,
              price: 1,
              review_scores_rating: 1,
            },
          },
        ],
        as: "b",
      },
    },
    // flatten the lookup result and ditch the wrapper so we return clean listing
```

```
docs
    { $unwind: "$b" },
    { $replaceRoot: { newRoot: "$b" } },
    // best-rated first, listing_id as tiebreaker so results are deterministic
    { $sort: { review_scores_rating: -1, listing_id: 1 } },
    { $limit: o.limit || 25 },
  ];
}

async function run(db, o) {
  const a = pipe(o);
  return db.collection("calendar").aggregate(a).toArray();
}

// same pipeline, but ask the planner to dump executionStats so the report can cite
real numbers
async function explain(db, o) {
  const a = pipe(o);
  return db.collection("calendar").aggregate(a).explain("executionStats");
}

module.exports = { run, explain, pipe };
```

src/q2.js

Q2 - neighborhoods with zero active listings in a given month

```
// q2: neighborhoods in <city> that had ZERO available listings during the given
month
// implemented as: positive aggregation (active set) + app-side anti-join against
the full nbhd set
// reason for the split: $lookup-based set-difference inside mongo would scan the
whole nbhd
// collection per active row, and the active set is small enough to diff in-process
trivially
function activePipe(o) {
  return [
    // calendar window narrowed by the (city, date, available) index
    { $match: { city: o.city, date: { $gte: o.from, $lte: o.to }, available: true }
},
    // collapse to one row per active listing first, so the lookup runs once per
listing not per day
    { $group: { _id: { c: "$city", l: "$listing_id" } } },
    // join to listings just to get the neighborhood field, hits the unique (city,
listing_id) index
    {
      $lookup: {
        from: "listings",
        let: { c: "$_id.c", l: "$_id.l" },
        pipeline: [
          { $match: { $expr: { $and: [{ $eq: ["$city", "$$c"] }, { $eq:
["$listing_id", "$$l"] }] } } },
          { $project: { _id: 0, neighborhood: 1 } },
        ],
        as: "n",
      },
    },
    { $unwind: "$n" },
    // distinct (city, neighborhood) -- this is the "active" set we'll subtract from
    { $group: { _id: { c: "$_id.c", n: "$n.neighborhood" } } },
  ];
}

// month string "YYYY-MM" -> [first, last] inclusive bounds; "31" works for every
month because
// $gte/$lte on a string date silently clamps -- no calendar row will have e.g. feb-
31
function mb(m) {
  const a = `${m}-01`;
  const b = `${m}-31`;
```

```
  return [a, b];
}

async function run(db, o) {
  const [a, b] = mb(o.month);
  const c = { city: o.city, from: a, to: b };
  // active neighborhoods this month
  const d = await db.collection("calendar").aggregate(activePipe(c)).toArray();
  const e = new Set(d.map((x) => x._id.n));
  // every known neighborhood in the city (the universe to subtract from)
  const f = await db
    .collection("neighborhoods")
    .find({ city: o.city }, { projection: { _id: 0, neighborhood: 1 } })
    .toArray();
  // anti-join: full set minus active set -> the "dark" neighborhoods
  const g = f
    .map((x) => x.neighborhood)
    .filter((x) => x && !e.has(x))
    .sort();
  return {
    city: o.city,
    month: o.month,
    total_neighborhoods: f.length,
    active_neighborhoods: e.size,
    empty_neighborhoods: g,
  };
}

async function explain(db, o) {
  const [a, b] = mb(o.month);
  const c = { city: o.city, from: a, to: b };
  return
db.collection("calendar").aggregate(activePipe(c)).explain("executionStats");
}

module.exports = { run, explain, activePipe };
```

src/q3.js

Q3 - per-listing bookable intervals, min-nights aware

```
// q3: for each entire-home listing in <city>, find every bookable interval inside
<month>
// "bookable" is stricter than "available": each day in the run must have enough
remaining
// consecutive available days to satisfy that day's minimum_nights (host's stay-
length rule)
// runs the interval scanner in app code -- mongo can't express this with
aggregation alone

// month string "YYYY-MM" -> [first, last] inclusive bounds; same trick as q2 (-31
always safe)
function mb(m) {
  return [`${m}-01`, `${m}-31`];
}

// true iff b is the calendar day immediately after a
// guards against gaps in the calendar (insideairbnb sometimes skips a day for a
listing)
function nx(a, b) {
  if (!a || !b) return false;
  const x = new Date(`${a}T00:00:00Z`);
  const y = new Date(`${b}T00:00:00Z`);
  return y.getTime() - x.getTime() === 86400000;
}

// core scanner: walk a listing's sorted calendar rows, output [{from, to,
min_nights}]
// algorithm:
//   1) skip unavailable days
//   2) j = end of the current run of consecutive available days
//   3) inside that run, keep day k only if remaining length (j-k) >= that day's
min_nights
//   4) collapse adjacent kept days back into [from, to] ranges
function intervals(r) {
  const o = [];
  let i = 0;
  while (i < r.length) {
    if (!r[i].available) {
      i++;
      continue;
    }
    let j = i + 1;
    while (j < r.length && r[j].available && nx(r[j - 1].date, r[j].date)) j++;
```

```
    // v = indices of days within [i, j) that pass the min-nights test
    const v = [];
    for (let k = i; k < j; k++) {
      const m = j - k;
      const n = r[k].minimum_nights || 1;
      if (m >= n) v.push(k);
    }
    // s/p track the start/end indices of the current "kept" run as we collapse
    let s = -1;
    let p = -1;
    for (let k = 0; k < v.length; k++) {
      const x = v[k];
      if (s === -1) {
        s = x;
        p = x;
      } else if (x === p + 1) {
        p = x;
      } else {
        o.push({ from: r[s].date, to: r[p].date, min_nights: r[s].minimum_nights });
        s = x;
        p = x;
      }
    }
    if (s !== -1) o.push({ from: r[s].date, to: r[p].date, min_nights:
r[s].minimum_nights });
    i = j;
  }
  return o;
}

// candidate listings narrowed by the (city, neighborhood, room_type) index prefix
on (city, room_type)
async function listingsFor(db, c, t) {
  return db
    .collection("listings")
    .find({ city: c, room_type: t }, { projection: { _id: 0, listing_id: 1, name: 1
} })
    .toArray();
}

async function run(db, o) {
  const c = o.city;
  const t = o.room_type || "Entire home/apt";
  const m = o.month;
  const [s, e] = mb(m);

  const a = await listingsFor(db, c, t);
  const b = a.map((x) => x.listing_id);
  if (!b.length) return [];
```

```
  // single bulk read of every relevant calendar row, sorted so the scanner can walk
in order
  // hits the (city, listing_id, date) index, projection keeps doc size small
  const k = await db
    .collection("calendar")
    .find(
      { city: c, listing_id: { $in: b }, date: { $gte: s, $lte: e } },
      { projection: { _id: 0, listing_id: 1, date: 1, available: 1, minimum_nights:
1 } }
    )
    .sort({ listing_id: 1, date: 1 })
    .toArray();

  // bucket calendar rows by listing so the scanner operates on each listing in
isolation
  const g = new Map();
  for (const r of k) {
    if (!g.has(r.listing_id)) g.set(r.listing_id, []);
    g.get(r.listing_id).push(r);
  }

  const out = [];
  for (const l of a) {
    const r = g.get(l.listing_id) || [];
    if (!r.length) continue;
    const iv = intervals(r);
    if (!iv.length) continue;
    out.push({ listing_id: l.listing_id, listing_name: l.name, month: m, intervals:
iv });
  }
  // most-bookable listings first, listing_id as tiebreaker for determinism
  out.sort((p, q) => q.intervals.length - p.intervals.length || p.listing_id -
q.listing_id);
  return out;
}

// explain only the indexed candidate-fetch step; the interval scanner is app-side
and not pipelined
async function explain(db, o) {
  const c = o.city;
  const t = o.room_type || "Entire home/apt";
  return db
    .collection("listings")
    .find({ city: c, room_type: t }, { projection: { _id: 0, listing_id: 1 } })
    .explain("executionStats");
}

module.exports = { run, explain, intervals };
```

src/q4.js

Q4 - portland mar-aug bookable-nights trend (reuses Q3)

```
// q4: portland entire-home booking-trend across march -> august of <year>
// composes q3's interval scanner across six months and totals bookable nights per
month
// this is the rare query where reuse across modules is exact -- no copy/paste of
the scanner
const H = require("./q3");

// fixed march..august window per the assignment's seasonal-demand framing
function months(y) {
  return ["03", "04", "05", "06", "07", "08"].map((m) => `${y}-${m}`);
}

// sum total nights across an array of {from, to} intervals, inclusive on both ends
// +1 because [from=01, to=03] is 3 nights, not 2
function totalNights(iv) {
  let n = 0;
  for (const x of iv) {
    const a = new Date(`${x.from}T00:00:00Z`);
    const b = new Date(`${x.to}T00:00:00Z`);
    n += Math.floor((b.getTime() - a.getTime()) / 86400000) + 1;
  }
  return n;
}

async function run(db, o) {
  const c = o.city || "portland";
  const t = o.room_type || "Entire home/apt";
  const y = o.year || "2025";
  const a = months(y);
  const r = [];
  // serial loop on purpose: each q3 run already pulls a sizeable per-listing
calendar batch,
  // and parallelising 6 of them blows out the in-memory mongo connection pool
  for (const m of a) {
    const k = await H.run(db, { city: c, room_type: t, month: m });
    let n = 0;
    let l = 0;
    for (const x of k) {
      n += totalNights(x.intervals);
      l += 1;
    }
    r.push({ month: m, listings_with_intervals: l, total_bookable_nights: n });
  }
```

```
  return { city: c, room_type: t, year: y, months: r };
}

// reuses q3's explain (the candidate-listing fetch) since that's the only mongo-
side step
async function explain(db, o) {
  const c = o.city || "portland";
  const t = o.room_type || "Entire home/apt";
  return H.explain(db, { city: c, room_type: t });
}

module.exports = { run, explain, totalNights };
```

src/q5.js

Q5 - december reviews per (city, year)

```
// q5: count of december reviews per (city, year) -- one pure aggregation over the
reviews collection
// no index helps here (we filter by month-of-date, not a prefix), so this is an
honest COLLSCAN
// kept as a scan deliberately to demonstrate the cost of querying on a derived date
component
function pipe() {
  return [
    // regex on YYYY-12-DD picks every december review across all years; full
collection scan
    { $match: { date: { $regex: /^\d{4}-12-\d{2}$/ } } },
    // group by (city, year-extracted-from-date); $substr is cheap once we're
already scanning
    {
      $group: {
        _id: { city: "$city", year: { $substr: ["$date", 0, 4] } },
        review_count: { $sum: 1 },
      },
    },
    // flatten the compound _id so callers get a clean shape
    {
      $project: {
        _id: 0,
        city: "$_id.city",
        year: "$_id.year",
        review_count: 1,
      },
    },
    { $sort: { city: 1, year: 1 } },
  ];
}

async function run(db) {
  return db.collection("reviews").aggregate(pipe()).toArray();
}

async function explain(db) {
  return db.collection("reviews").aggregate(pipe()).explain("executionStats");
}

module.exports = { run, explain, pipe };
```

src/q6.js

Q6 - re-book reminders + same-host listings

```
// q6: re-book reminders -- for every (city, listing, reviewer) the same guest
reviewed twice+,
// surface the listing, the host's other listings in the same city, and current
calendar status
// designed as one aggregation + a small batch of indexed point reads per row, NOT
one giant
// nested $lookup pipeline -- the latter blows out memory on 3M reviews and is
harder to explain

// stage 1: pull every (city, listing, reviewer) combo with >1 review
// hits the (reviewer_id, listing_id, date) index prefix
function repeatPipe() {
  return [
    { $match: { reviewer_id: { $ne: null } } },
    {
      $group: {
        _id: { city: "$city", listing_id: "$listing_id", reviewer_id: "$reviewer_id"
},
        review_dates: { $push: "$date" },
        review_count: { $sum: 1 },
      },
    },
    { $match: { review_count: { $gt: 1 } } },
    { $sort: { "_id.city": 1, "_id.listing_id": 1 } },
  ];
}

// review date "YYYY-MM-DD" -> [first, last] of that month; same -31 trick as q2/q3
function mb(d) {
  const a = d.substring(0, 7);
  const b = `${a}-01`;
  const c = `${a}-31`;
  return [b, c];
}

async function run(db, o) {
  const a = await db.collection("reviews").aggregate(repeatPipe()).toArray();
  const b = [];
  const c = o && o.limit ? o.limit : 25;

  // per repeat-pair: 4 indexed reads max (listing point lookup, calendar day check,
  // calendar min/max aggregate, same-host listings find) -- cheap because each is
index-driven
```

```
  for (const g of a) {
    if (b.length >= c) break;

    // listing point lookup via unique (city, listing_id) index
    const h = await db.collection("listings").findOne(
      { city: g._id.city, listing_id: g._id.listing_id },
      { projection: { _id: 0, name: 1, listing_url: 1, description: 1, host_id: 1,
host_name: 1 } }
    );
    if (!h) continue;

    // one reminder row per review-date this guest left -- preserves the time
dimension
    for (const d of g.review_dates) {
      if (!d) continue;
      const [s, e] = mb(d);

      // any availability in the same month as the review? hits the (city,
listing_id, date) index
      const k = await db.collection("calendar").findOne({
        city: g._id.city,
        listing_id: g._id.listing_id,
        date: { $gte: s, $lte: e },
        available: true,
      });

      // pull min/max stay rules from the same calendar window for display
      const m = await db
        .collection("calendar")
        .aggregate([
          { $match: { city: g._id.city, listing_id: g._id.listing_id, date: { $gte:
s, $lte: e } } },
          { $group: { _id: null, mn: { $min: "$minimum_nights" }, mx: { $max:
"$maximum_nights" } } },
        ])
        .toArray();

      // host's OTHER listings in the same city, capped at 10 for display
      // uses the (host_id, city) index -- key order critiqued in the report (city
should come first)
      const q = await db
        .collection("listings")
        .find(
          { city: g._id.city, host_id: h.host_id, listing_id: { $ne:
g._id.listing_id } },
          { projection: { _id: 0, name: 1, listing_url: 1 } }
        )
        .limit(10)
        .toArray();
```

```
      b.push({
        listing_name: h.name,
        listing_url: h.listing_url,
        description: h.description,
        host_name: h.host_name,
        reviewer_id: g._id.reviewer_id,
        previously_booked: true,
        month: d.substring(0, 7),
        min_nights: m.length ? m[0].mn : null,
        max_nights: m.length ? m[0].mx : null,
        available_in_same_month: !!k,
        other_host_listings_same_city: q,
      });

      if (b.length >= c) break;
    }
  }

  return b;
}

// explain only the headline aggregation -- the per-row point reads are tracked by
their own indexes
async function explain(db) {
  return
db.collection("reviews").aggregate(repeatPipe()).explain("executionStats");
}

module.exports = { run, explain, repeatPipe };
```

src/driver.js

CLI runner: boots mongo, loads data, runs all 6 queries + explain

```
// driver: end-to-end CLI runner
// boots an in-process mongo, calls the loader, picks demo dates from the loaded
data,
// runs all 6 queries, captures explain('executionStats') for each, and writes
everything to out/
// (if you only want the demo UI, run web/server.js instead -- this script is for
repro / grading)
const fs = require("fs/promises");
const path = require("path");
const { MongoMemoryServer } = require("mongodb-memory-server");
const { MongoClient } = require("mongodb");

const A = require("./loader");
const B = require("./q1");
const C = require("./q5");
const D = require("./q6");
const G = require("./q2");
const H = require("./q3");
const I = require("./q4");

const O = path.join(__dirname, "..", "out");

// add n days to a yyyy-mm-dd string, returning yyyy-mm-dd; UTC math avoids DST
surprises
function ds(s, n) {
  const d = new Date(`${s}T00:00:00Z`);
  d.setUTCDate(d.getUTCDate() + n);
  return d.toISOString().substring(0, 10);
}

// q1 needs two consecutive days where SOMETHING is available in portland
// without this the demo could pick a window with zero matches, which looks broken
async function pickPortlandDates(db) {
  const a = await db
    .collection("calendar")
    .aggregate([
      { $match: { city: "portland", available: true } },
      { $group: { _id: "$date", c: { $sum: 1 } } },
      { $match: { c: { $gte: 1 } } },
      { $sort: { _id: 1 } },
    ])
    .toArray();
```

```
  // walk the sorted active-day list, return the first pair where day+1 also exists
  for (let i = 0; i < a.length; i++) {
    const s = a[i]._id;
    const e = ds(s, 1);
    if (a.find((x) => x._id === e)) return [s, e];
  }
  return [a[0]._id, a[Math.min(1, a.length - 1)]._id];
}

// q2/q3 want a month that's actually present in the loaded calendar
// pick the middle one so the demo doesn't sit at the edge of the snapshot window
async function pickMonth(db, c) {
  const a = await db
    .collection("calendar")
    .aggregate([
      { $match: { city: c } },
      { $group: { _id: { $substr: ["$date", 0, 7] }, n: { $sum: 1 } } },
      { $sort: { _id: 1 } },
    ])
    .toArray();
  return a.length ? a[Math.floor(a.length / 2)]._id : null;
}

// pretty-print a collection's indexes for the load_evidence dump
async function fmtIdx(c) {
  const a = await c.indexes();
  return a
    .filter((i) => i.name !== "_id_")
    .map((i) => `  { ${Object.keys(i.key).join(", ")} }${i.unique ? " [unique]" :
""}`)
    .join("\n");
}

// per-city counts for the load evidence file (sanity check that each city loaded)
async function perCity(db, k) {
  const a = ["los_angeles", "portland", "salem", "san_diego"];
  const r = {};
  for (const c of a) {
    r[c] = await db.collection(k).countDocuments({ city: c });
  }
  return r;
}

// dumps load_evidence.txt -- this is the file referenced from the report's loader
section
// includes total counts, per-city counts, sample docs, and the index list, so a
grader can
// verify the schema/index claims without running the loader themselves
async function writeEvidence(db, counts, dates, m2, m3, m4) {
  const a = await db.collection("listings").find({ city: "portland" }, { projection:
```

```
{ _id: 0, listing_id: 1, name: 1, neighborhood: 1, room_type: 1, price: 1,
review_scores_rating: 1, host_name: 1 } }).limit(3).toArray();
  const b = await db.collection("calendar").find({ city: "salem" }, { projection: {
_id: 0 } }).limit(3).toArray();
  const c = await db.collection("reviews").find({ city: "san_diego" }, { projection:
{ _id: 0, comments: 0 } }).limit(3).toArray();
  const d = await db.collection("neighborhoods").find({ city: "los_angeles" }, {
projection: { _id: 0 } }).limit(4).toArray();

  const pcL = await perCity(db, "listings");
  const pcR = await perCity(db, "reviews");
  const pcC = await perCity(db, "calendar");
  const pcN = await perCity(db, "neighborhoods");

  const fmt = (o) => Object.entries(o).map(([k, v]) => `    ${k.padEnd(14)}
${v.toLocaleString()}`).join("\n");

  const e = [
    "$ npm start  (mongodb-memory-server)",
    "",
    `Connected DB: ${db.databaseName}`,
    "",
    "Collection counts (loaded):",
    `  listings:       ${counts.listings.toLocaleString()}`,
    `  reviews:        ${counts.reviews.toLocaleString()}`,
    `  neighborhoods:  ${counts.neighborhoods.toLocaleString()}`,
    `  calendar:       ${counts.calendar.toLocaleString()}`,
    "",
    "Per-city listings:",
    fmt(pcL),
    "Per-city reviews:",
    fmt(pcR),
    "Per-city calendar:",
    fmt(pcC),
    "Per-city neighborhoods:",
    fmt(pcN),
    "",
    `Q1 demo dates picked from data: ${dates[0]} -> ${dates[1]}`,
    `Q2 demo month: ${m2}`,
    `Q3 demo month: ${m3}`,
    `Q4 demo year:  ${m4}`,
    "",
    "Sample listings (Portland):",
    ...a.map((x) => JSON.stringify(x, null, 2)),
    "",
    "Sample calendar rows (Salem):",
    ...b.map((x) => JSON.stringify(x, null, 2)),
    "",
    "Sample reviews (San Diego):",
    ...c.map((x) => JSON.stringify(x, null, 2)),
```

```
    "",
    "Sample neighborhoods (Los Angeles):",
    ...d.map((x) => JSON.stringify(x, null, 2)),
    "",
    "Indexes on listings:",
    await fmtIdx(db.collection("listings")),
    "",
    "Indexes on calendar:",
    await fmtIdx(db.collection("calendar")),
    "",
    "Indexes on reviews:",
    await fmtIdx(db.collection("reviews")),
    "",
    "Indexes on neighborhoods:",
    await fmtIdx(db.collection("neighborhoods")),
  ].join("\n");

  await fs.writeFile(path.join(O, "load_evidence.txt"), e, "utf-8");
}

async function main() {
  await fs.mkdir(O, { recursive: true });
  console.log("[driver] starting mongodb-memory-server");
  // pin the binary to 8.0.3 and reuse the cached download dir from task3-
deliverables/
  // so re-runs don't have to redownload the ~150MB mongo binary
  const m = await MongoMemoryServer.create({
    binary: { version: "8.0.3", downloadDir: path.join(__dirname, "..", "..",
"task3-deliverables", "data", "mongodb-binaries") },
  });
  const u = m.getUri();
  console.log(`[driver] uri ${u}`);
  const c = new MongoClient(u);
  await c.connect();
  const db = c.db("airbnb_stage3");

  try {
    console.log("[driver] loading data");
    const counts = await A.load(db);
    console.log("[driver] counts", counts);

    // pick demo parameters AFTER load so they always match the data that's actually
present
    const dates = await pickPortlandDates(db);
    console.log(`[driver] Q1 dates ${dates[0]} ${dates[1]}`);

    const m2 = await pickMonth(db, "portland");
    const m3 = await pickMonth(db, "salem");
    // q4 just needs the year of the chosen month -- months() inside q4.js fixes
mar..aug
```

```
    const m4 = (m2 || "2025-12").substring(0, 4);
    console.log(`[driver] Q2 month ${m2}, Q3 month ${m3}, Q4 year ${m4}`);

    await writeEvidence(db, counts, dates, m2, m3, m4);

    // each query: run -> dump results json, explain -> dump executionStats json
    // these json files back the numbers cited in the report's section 3
    console.log("[driver] running Q1");
    const q1 = await B.run(db, { city: "portland", dates, limit: 25 });
    const q1e = await B.explain(db, { city: "portland", dates, limit: 25 });
    await fs.writeFile(path.join(O, "q1_results.json"), JSON.stringify(q1, null,
2));
    await fs.writeFile(path.join(O, "explain_q1.json"), JSON.stringify(q1e, null,
2));
    console.log(`[driver] Q1 returned ${q1.length} rows`);

    console.log("[driver] running Q2");
    const q2 = await G.run(db, { city: "portland", month: m2 });
    const q2e = await G.explain(db, { city: "portland", month: m2 });
    await fs.writeFile(path.join(O, "q2_results.json"), JSON.stringify(q2, null,
2));
    await fs.writeFile(path.join(O, "explain_q2.json"), JSON.stringify(q2e, null,
2));
    console.log(`[driver] Q2
${q2.empty_neighborhoods.length}/${q2.total_neighborhoods} neighborhoods empty in
${m2}`);

    console.log("[driver] running Q3");
    const q3 = await H.run(db, { city: "salem", room_type: "Entire home/apt",
month: m3 });
    const q3e = await H.explain(db, { city: "salem", room_type: "Entire home/apt"
});
    await fs.writeFile(path.join(O, "q3_results.json"), JSON.stringify(q3, null,
2));
    await fs.writeFile(path.join(O, "explain_q3.json"), JSON.stringify(q3e, null,
2));
    console.log(`[driver] Q3 ${q3.length} listings with bookable intervals in
${m3}`);

    console.log("[driver] running Q4");
    const q4 = await I.run(db, { city: "portland", room_type: "Entire home/apt",
year: m4 });
    const q4e = await I.explain(db, { city: "portland", room_type: "Entire home/apt"
});
    await fs.writeFile(path.join(O, "q4_results.json"), JSON.stringify(q4, null,
2));
    await fs.writeFile(path.join(O, "explain_q4.json"), JSON.stringify(q4e, null,
2));
    console.log(`[driver] Q4 totals per month`, q4.months.map((x) =>
`${x.month}=${x.total_bookable_nights}`).join(" "));
```

```
    console.log("[driver] running Q5");
    const q5 = await C.run(db);
    const q5e = await C.explain(db);
    await fs.writeFile(path.join(O, "q5_results.json"), JSON.stringify(q5, null,
2));
    await fs.writeFile(path.join(O, "explain_q5.json"), JSON.stringify(q5e, null,
2));
    console.log(`[driver] Q5 returned ${q5.length} rows`);

    console.log("[driver] running Q6");
    const q6 = await D.run(db, { limit: 25 });
    const q6e = await D.explain(db);
    await fs.writeFile(path.join(O, "q6_results.json"), JSON.stringify(q6, null,
2));
    await fs.writeFile(path.join(O, "explain_q6.json"), JSON.stringify(q6e, null,
2));
    console.log(`[driver] Q6 returned ${q6.length} rows`);

    // single-line-per-query rollup, useful for quick eyeballing without opening
every json
    const s = {
      counts,
      q1: { rows: q1.length, dates },
      q2: { city: "portland", month: m2, empty: q2.empty_neighborhoods.length,
total: q2.total_neighborhoods },
      q3: { city: "salem", month: m3, listings_with_intervals: q3.length },
      q4: { city: "portland", year: m4, months: q4.months },
      q5: { rows: q5.length },
      q6: { rows: q6.length },
    };
    await fs.writeFile(path.join(O, "summary.json"), JSON.stringify(s, null, 2));
    console.log("[driver] done", s);
  } finally {
    // always release the mongo binary subprocess so re-running doesn't leak ports
    await c.close();
    await m.stop();
  }
}

main().catch((e) => {
  console.error(e);
  process.exit(1);
});
```

web/server.js

express server exposing each query as an HTTP endpoint

```
// web server: same in-process mongo + same loader as the CLI driver, but exposes
the queries
// as HTTP endpoints behind a static frontend in web/public/
// boot is async -- /api/health gates the frontend until S.ready flips true
const path = require("path");
const express = require("express");
const { MongoMemoryServer } = require("mongodb-memory-server");
const { MongoClient } = require("mongodb");

const A = require("../src/loader");
const B = require("../src/q1");
const C = require("../src/q5");
const D = require("../src/q6");
const G = require("../src/q2");
const H = require("../src/q3");
const I = require("../src/q4");

const P = process.env.PORT ? Number(process.env.PORT) : 4173;

// shared boot state, the frontend polls /api/health and watches S.ready
const S = {
  ready: false,
  err: null,
  counts: null,
  dates: null,
  startedAt: Date.now(),
};

// module-level singletons for the mongo handles -- re-using them across requests
// (creating a new MongoClient per request would re-handshake on every API call)
let G_DB = null;
let G_CLIENT = null;
let G_MEM = null;

// add n days to YYYY-MM-DD (UTC math, see driver for the same helper)
function ds(s, n) {
  const d = new Date(`${s}T00:00:00Z`);
  d.setUTCDate(d.getUTCDate() + n);
  return d.toISOString().substring(0, 10);
}

// q1 default-date helper, same logic as the driver -- finds two consecutive
available dates
```

```
async function pickDates(db, c) {
  const a = await db
    .collection("calendar")
    .aggregate([
      { $match: { city: c, available: true } },
      { $group: { _id: "$date", n: { $sum: 1 } } },
      { $sort: { _id: 1 } },
    ])
    .toArray();

  for (let i = 0; i < a.length; i++) {
    const s = a[i]._id;
    const e = ds(s, 1);
    if (a.find((x) => x._id === e)) return [s, e];
  }
  return a.length ? [a[0]._id, a[Math.min(1, a.length - 1)]._id] : null;
}

// q2/q3 default-month helper, same as driver -- middle month of the loaded calendar
async function pickMonth(db, c) {
  const a = await db
    .collection("calendar")
    .aggregate([
      { $match: { city: c } },
      { $group: { _id: { $substr: ["$date", 0, 7] }, n: { $sum: 1 } } },
      { $sort: { _id: 1 } },
    ])
    .toArray();
  return a.length ? a[Math.floor(a.length / 2)]._id : null;
}

// run once at startup: spin mongo, load all four collections, pre-pick demo
defaults
// listening on the port happens immediately (see app.listen below) so /api/health
can answer
// "still loading" instead of refusing connections
async function boot() {
  console.log("[web] starting mongodb-memory-server");
  G_MEM = await MongoMemoryServer.create({
    binary: { version: "8.0.3", downloadDir: path.join(__dirname, "..", "..",
"task3-deliverables", "data", "mongodb-binaries") },
  });
  const u = G_MEM.getUri();
  console.log(`[web] uri ${u}`);
  G_CLIENT = new MongoClient(u);
  await G_CLIENT.connect();
  G_DB = G_CLIENT.db("airbnb_stage3");

  console.log("[web] loading data");
  S.counts = await A.load(G_DB);
```

```
  S.dates = await pickDates(G_DB, "portland");
  S.month_portland = await pickMonth(G_DB, "portland");
  S.month_salem = await pickMonth(G_DB, "salem");
  S.year_default = (S.month_portland || "2025-12").substring(0, 4);
  S.ready = true;
  console.log("[web] ready", { counts: S.counts, q1_dates: S.dates, q2_month:
S.month_portland, q3_month: S.month_salem, q4_year: S.year_default });
}

// fire-and-forget; we still want app.listen to start so frontend gets boot status,
not a connection refused
boot().catch((e) => {
  console.error("[web] boot failed", e);
  S.err = String(e && e.message ? e.message : e);
});

const app = express();
app.use(express.json());
app.use(express.static(path.join(__dirname, "public")));

// frontend polls this until ready=true; also exposes pre-picked demo defaults so
the UI can
// render hints like "suggested month: 2025-09" before the user hits any query
app.get("/api/health", (_req, res) => {
  res.json({
    ready: S.ready,
    err: S.err,
    counts: S.counts,
    q1_default_dates: S.dates,
    q2_default_month: S.month_portland,
    q3_default_month: S.month_salem,
    q4_default_year: S.year_default,
    uptime_ms: Date.now() - S.startedAt,
  });
});

// every query route runs through this guard so requests during boot get a 503
instead of a crash
function gate(req, res) {
  if (!S.ready) {
    res.status(503).json({ error: "data still loading", ready: false });
    return false;
  }
  return true;
}

app.get("/api/q1", async (req, res) => {
  if (!gate(req, res)) return;
  try {
    const c = (req.query.city || "portland").toString();
```

```
    const a = req.query.from ? req.query.from.toString() : null;
    const b = req.query.to ? req.query.to.toString() : null;
    let d = null;
    // user-provided window OR auto-pick a known-good pair from the loaded data
    if (a && b) d = [a, b];
    else d = await pickDates(G_DB, c);
    if (!d) return res.json({ city: c, dates: null, results: [] });
    const l = req.query.limit ? Number(req.query.limit) : 25;
    const r = await B.run(G_DB, { city: c, dates: d, limit: l });
    res.json({ city: c, dates: d, results: r });
  } catch (e) {
    res.status(500).json({ error: String(e && e.message ? e.message : e) });
  }
});

// q5 takes no params -- it's a global aggregation over the entire reviews
collection
app.get("/api/q5", async (_req, res) => {
  if (!gate(_req, res)) return;
  try {
    const r = await C.run(G_DB);
    res.json({ results: r });
  } catch (e) {
    res.status(500).json({ error: String(e && e.message ? e.message : e) });
  }
});

app.get("/api/q6", async (req, res) => {
  if (!gate(req, res)) return;
  try {
    const l = req.query.limit ? Number(req.query.limit) : 25;
    const r = await D.run(G_DB, { limit: l });
    res.json({ results: r });
  } catch (e) {
    res.status(500).json({ error: String(e && e.message ? e.message : e) });
  }
});

app.get("/api/q2", async (req, res) => {
  if (!gate(req, res)) return;
  try {
    const c = (req.query.city || "portland").toString();
    // fall back to the pre-picked demo month so the UI always returns SOMETHING on
first click
    const m = (req.query.month || S.month_portland || "2025-12").toString();
    const r = await G.run(G_DB, { city: c, month: m });
    res.json(r);
  } catch (e) {
    res.status(500).json({ error: String(e && e.message ? e.message : e) });
  }
```

```
});

app.get("/api/q3", async (req, res) => {
  if (!gate(req, res)) return;
  try {
    const c = (req.query.city || "salem").toString();
    const m = (req.query.month || S.month_salem || S.month_portland || "2025-
12").toString();
    const t = (req.query.room_type || "Entire home/apt").toString();
    const r = await H.run(G_DB, { city: c, room_type: t, month: m });
    res.json({ city: c, month: m, room_type: t, results: r });
  } catch (e) {
    res.status(500).json({ error: String(e && e.message ? e.message : e) });
  }
});

app.get("/api/q4", async (req, res) => {
  if (!gate(req, res)) return;
  try {
    const c = (req.query.city || "portland").toString();
    const y = (req.query.year || S.year_default || "2025").toString();
    const t = (req.query.room_type || "Entire home/apt").toString();
    const r = await I.run(G_DB, { city: c, year: y, room_type: t });
    res.json(r);
  } catch (e) {
    res.status(500).json({ error: String(e && e.message ? e.message : e) });
  }
});

app.listen(P, () => {
  console.log(`[web] listening on http://localhost:${P}`);
});

// SIGINT/SIGTERM handlers so the mongo subprocess gets reaped on Ctrl+C
async function shutdown() {
  console.log("[web] shutdown");
  try {
    if (G_CLIENT) await G_CLIENT.close();
  } catch (e) {}
  try {
    if (G_MEM) await G_MEM.stop();
  } catch (e) {}
  process.exit(0);
}
process.on("SIGINT", shutdown);
process.on("SIGTERM", shutdown);
```

web/public/index.html

demo UI markup -- one tab per query

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="utf-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1" />
    <title>AirBnB &times; MongoDB &mdash; Stage 3 Prototype</title>
    <link rel="preconnect" href="https://fonts.googleapis.com" />
    <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin />
    <link href="https://fonts.googleapis.com/css2?
family=Inter:wght@400;500;600;700;800&display=swap" rel="stylesheet" />
    <link rel="stylesheet" href="styles.css" />
  </head>
  <body>
    <!-- boot overlay is shown on first paint, hidden by app.js once /api/health
flips ready=true -->
    <div class="boot" id="boot">
      <div class="boot-card">
        <div class="boot-spinner"></div>
        <h2>Booting in-memory MongoDB&hellip;</h2>
        <p id="boot-status">Spinning up <code>mongodb-memory-server</code>,
fetching CSVs from <code>data.insideairbnb.com</code>, building 8 indexes. First
boot takes ~10 seconds.</p>
      </div>
    </div>

    <!-- topbar: brand on the left, live per-collection counts on the right (filled
from /api/health) -->
    <header class="topbar">
      <div class="brand">
        <div class="brand-mark">m</div>
        <div class="brand-text">
          <div class="brand-title">AirBnB &middot; MongoDB Explorer</div>
          <div class="brand-sub">CS 498 Stage 3 prototype &middot; in-
memory cluster</div>
        </div>
      </div>
      <div class="stats" id="stats">
        <div class="stat"><span class="stat-label">listings</span><span
class="stat-value" id="s-l">&mdash;</span></div>
        <div class="stat"><span class="stat-label">calendar</span><span
class="stat-value" id="s-c">&mdash;</span></div>
        <div class="stat"><span class="stat-label">reviews</span><span class="stat-
value" id="s-r">&mdash;</span></div>
```

```
        <div class="stat"><span class="stat-label">neighborhoods</span><span
class="stat-value" id="s-n">&mdash;</span></div>
      </div>
    </header>

    <!-- one tab per query; data-tab is read by app.js to toggle the matching panel
below -->
    <nav class="tabs">
      <button class="tab is-active" data-tab="q1">
        <span class="tab-tag">Q1</span>
        <span class="tab-title">Two-night search</span>
        <span class="tab-sub">filter &middot; group &middot; lookup &middot;
sort</span>
      </button>
      <button class="tab" data-tab="q2">
        <span class="tab-tag">Q2</span>
        <span class="tab-title">Empty neighborhoods</span>
        <span class="tab-sub">anti-join &middot; per city &middot; per month</span>
      </button>
      <button class="tab" data-tab="q3">
        <span class="tab-tag">Q3</span>
        <span class="tab-title">Salem availability windows</span>
        <span class="tab-sub">interval scan &middot; min-nights aware</span>
      </button>
      <button class="tab" data-tab="q4">
        <span class="tab-tag">Q4</span>
        <span class="tab-title">Portland booking trend</span>
        <span class="tab-sub">monthly bookable nights &middot; Mar&ndash;Aug</span>
      </button>
      <button class="tab" data-tab="q5">
        <span class="tab-tag">Q5</span>
        <span class="tab-title">December reviews</span>
        <span class="tab-sub">aggregation &middot; per city &middot; per
year</span>
      </button>
      <button class="tab" data-tab="q6">
        <span class="tab-tag">Q6</span>
        <span class="tab-title">Re-book reminder</span>
        <span class="tab-sub">multi-step &middot; same-host listings</span>
      </button>
    </nav>

    <!-- panels: only one is .is-active at a time, app.js toggles via the tab data-
tab attribute -->
    <main class="content">
      <section id="panel-q1" class="panel is-active">
        <header class="panel-head">
          <div>
            <h1>Find a place for two consecutive nights</h1>
            <p class="lead">Pick a city and we&rsquo;ll surface the highest-rated
```

```
listings whose <code>calendar</code> is open for both dates.</p>
          </div>
        </header>

        <form class="search" id="q1-form">
          <label class="field">
            <span>City</span>
            <select id="q1-city">
              <option value="portland">Portland</option>
              <option value="los_angeles">Los Angeles</option>
              <option value="san_diego">San Diego</option>
              <option value="salem">Salem</option>
            </select>
          </label>
          <label class="field">
            <span>Check-in</span>
            <input type="date" id="q1-from" />
          </label>
          <label class="field">
            <span>Check-out</span>
            <input type="date" id="q1-to" />
          </label>
          <label class="field field-num">
            <span>Limit</span>
            <input type="number" id="q1-limit" min="1" max="100" value="12" />
          </label>
          <button type="submit" class="btn-primary">Search</button>
        </form>
        <div class="hint" id="q1-hint">Leave dates blank to use the earliest pair
of consecutive available dates from the loaded data.</div>

        <div class="results" id="q1-results"></div>
      </section>

      <section id="panel-q2" class="panel">
        <header class="panel-head">
          <div>
            <h1>Neighborhoods with no listings in a month</h1>
            <p class="lead">For a chosen city and month we anti-join
<code>calendar</code> activity against the full <code>neighborhoods</code> set.
Empty &equiv; nobody had a single available night.</p>
          </div>
        </header>
        <form class="search" id="q2-form">
          <label class="field">
            <span>City</span>
            <select id="q2-city">
              <option value="portland">Portland</option>
              <option value="los_angeles">Los Angeles</option>
              <option value="san_diego">San Diego</option>
```

```
              <option value="salem">Salem</option>
            </select>
          </label>
          <label class="field">
            <span>Month</span>
            <input type="month" id="q2-month" />
          </label>
          <button type="submit" class="btn-primary">Run query</button>
        </form>
        <div class="hint" id="q2-hint">Leave blank to use the loaded data&rsquo;s
middle month.</div>
        <div class="results" id="q2-results"></div>
      </section>

      <section id="panel-q3" class="panel">
        <header class="panel-head">
          <div>
            <h1>Salem entire-home bookable windows</h1>
            <p class="lead">For each Salem entire-home listing we scan its
<code>calendar</code> for the chosen month, then keep only contiguous runs whose
remaining length still satisfies that day&rsquo;s <code>minimum_nights</code>.</p>
          </div>
        </header>
        <form class="search" id="q3-form">
          <label class="field">
            <span>City</span>
            <select id="q3-city">
              <option value="salem">Salem</option>
              <option value="portland">Portland</option>
              <option value="san_diego">San Diego</option>
              <option value="los_angeles">Los Angeles</option>
            </select>
          </label>
          <label class="field">
            <span>Month</span>
            <input type="month" id="q3-month" />
          </label>
          <button type="submit" class="btn-primary">Find windows</button>
        </form>
        <div class="hint" id="q3-hint">Each card is one listing; date ranges inside
it are valid check-in&hellip;check-out windows.</div>
        <div class="results" id="q3-results"></div>
      </section>

      <section id="panel-q4" class="panel">
        <header class="panel-head">
          <div>
            <h1>Portland booking trend &mdash; March through August</h1>
            <p class="lead">Reuses the Q3 interval logic on a loop: per month, sum
every entire-home listing&rsquo;s bookable nights. Useful for spotting demand shape
```

```
across the season.</p>
          </div>
          <button id="q4-run" class="btn-primary">Run trend</button>
        </header>
        <div class="results" id="q4-results"></div>
      </section>

      <section id="panel-q5" class="panel">
        <header class="panel-head">
          <div>
            <h1>December reviews per city, per year</h1>
            <p class="lead">A pure aggregation over <code>reviews</code> &mdash;
one bar per (city, year) where <code>date</code> matches <code>YYYY-12-DD</code>.
</p>
          </div>
          <button id="q5-run" class="btn-primary">Run aggregation</button>
        </header>
        <div class="results" id="q5-results"></div>
      </section>

      <section id="panel-q6" class="panel">
        <header class="panel-head">
          <div>
            <h1>Reminders for repeat guests</h1>
            <p class="lead">For every reviewer who reviewed the same listing more
than once we look up its host&rsquo;s other listings in the same city &mdash; one
aggregation, then a small batch of indexed point reads.</p>
          </div>
          <div class="panel-controls">
            <label class="field-inline">
              <span>Limit</span>
              <input type="number" id="q6-limit" min="1" max="100" value="12" />
            </label>
            <button id="q6-run" class="btn-primary">Run query</button>
          </div>
        </header>
        <div class="results" id="q6-results"></div>
      </section>
    </main>

    <footer class="foot">
      <span>Indexes verified with <code>explain('executionStats')</code> &middot;
results in <code>stage3-prototype/out/</code></span>
      <a href="https://github.com/abachu2005/cs498-airbnb-mongodb-prototype"
target="_blank" rel="noopener">GitHub repository &rarr;</a>
    </footer>

    <script src="app.js"></script>
  </body>
</html>
```

web/public/app.js

demo UI client logic -- runQ/renderQ pair per query

```
// frontend: vanilla JS, no framework -- the entire app is one DOM tree + 6 fetch
handlers
// each Q has a runQ + renderQ pair: runQ fires the API call, renderQ paints the
result panel
// W tracks which tabs have been visited so we don't re-query on every tab click

// $ / $$ are 4-char shortcuts for querySelector / querySelectorAll, used everywhere
const $ = (s) => document.querySelector(s);
const $$ = (s) => Array.from(document.querySelectorAll(s));

// number formatter, "—" for null/undefined, locale-grouped commas otherwise
const F = (n) => (n == null ? "—" : n.toLocaleString());

// html-escape user-controlled strings before injecting into innerHTML (xss guard)
function esc(s) {
  if (s == null) return "";
  return String(s).replace(/[&<>"]/g, (c) => ({ "&": "&amp;", "<": "&lt;", ">":
"&gt;", '"': "&quot;" }[c]));
}

// snake_case city tag -> "Title Case", used for headers since the DB stores cities
as e.g. "san_diego"
function pretty(s) {
  if (!s) return "";
  return String(s)
    .split("_")
    .map((w) => w.charAt(0).toUpperCase() + w.slice(1))
    .join(" ");
}

// amenities are stored as a stringified JSON array, parse + cap at 4 for badge
display
function ams(a) {
  if (!a) return [];
  try {
    return JSON.parse(a).slice(0, 4);
  } catch (e) {
    return [];
  }
}

let H = null;
```

```
// boot poll: hammer /api/health every 800ms until the loader finishes, then unhide
the UI
async function poll() {
  try {
    const r = await fetch("/api/health");
    const j = await r.json();
    if (j.ready) {
      H = j;
      onReady(j);
      return;
    }
    if (j.err) {
      $("#boot-status").innerHTML = `<span style="color:#e0224b">Boot failed:
${esc(j.err)}</span>`;
      return;
    }
  } catch (e) {}
  setTimeout(poll, 800);
}

// once data is loaded: hide boot overlay, fill the per-collection counts in the
topbar,
// seed Q1/Q2/Q3 placeholder dates from the demo defaults the server pre-picked,
then auto-run Q1
function onReady(j) {
  $("#boot").classList.add("is-hidden");
  if (j.counts) {
    $("#s-l").textContent = F(j.counts.listings);
    $("#s-c").textContent = F(j.counts.calendar);
    $("#s-r").textContent = F(j.counts.reviews);
    $("#s-n").textContent = F(j.counts.neighborhoods);
  }
  if (j.q1_default_dates && j.q1_default_dates.length === 2) {
    $("#q1-from").placeholder = j.q1_default_dates[0];
    $("#q1-to").placeholder = j.q1_default_dates[1];
    $("#q1-hint").innerHTML = `Suggested window from loaded data:
<strong>${esc(j.q1_default_dates[0])} → ${esc(j.q1_default_dates[1])}</strong>.
Leave the date inputs blank to use it.`;
  }
  if (j.q2_default_month) {
    $("#q2-month").placeholder = j.q2_default_month;
    $("#q2-hint").innerHTML = `Suggested month from loaded data:
<strong>${esc(j.q2_default_month)}</strong>. Leave blank to use it.`;
  }
  if (j.q3_default_month) {
    $("#q3-month").placeholder = j.q3_default_month;
    $("#q3-hint").innerHTML = `Suggested month from loaded data:
<strong>${esc(j.q3_default_month)}</strong>. Each card is one Salem entire-home
listing.`;
  }
```

```
  runQ1();
}

// tab switching + lazy-run-on-first-click
// W is the "have we already run this tab once" flag set; auto-runs only on FIRST
visit
$$(".tab").forEach((b) =>
  b.addEventListener("click", () => {
    const t = b.dataset.tab;
    $$(".tab").forEach((x) => x.classList.toggle("is-active", x === b));
    $$(".panel").forEach((p) => p.classList.toggle("is-active", p.id ===
`panel-${t}`));
    if (t === "q2" && !W.q2) runQ2();
    if (t === "q3" && !W.q3) runQ3();
    if (t === "q4" && !W.q4) runQ4();
    if (t === "q5" && !W.q5) runQ5();
    if (t === "q6" && !W.q6) runQ6();
  })
);

// "have we run this tab at least once" -- prevents re-firing the (slow) Q5/Q6 query
on every click
const W = { q2: false, q3: false, q4: false, q5: false, q6: false };

$("#q1-form").addEventListener("submit", (e) => {
  e.preventDefault();
  runQ1();
});
$("#q2-form").addEventListener("submit", (e) => {
  e.preventDefault();
  runQ2();
});
$("#q3-form").addEventListener("submit", (e) => {
  e.preventDefault();
  runQ3();
});
$("#q4-run").addEventListener("click", runQ4);
$("#q5-run").addEventListener("click", runQ5);
$("#q6-run").addEventListener("click", runQ6);

// q1: highest-rated listings open for two consecutive nights -- airbnb-card style
grid
async function runQ1() {
  const c = $("#q1-city").value;
  const a = $("#q1-from").value;
  const b = $("#q1-to").value;
  const l = $("#q1-limit").value || 12;
  const u = new URLSearchParams({ city: c, limit: l });
  if (a) u.set("from", a);
  if (b) u.set("to", b);
```

```
  const o = $("#q1-results");
  o.innerHTML = `<div class="loading">Querying calendar &amp; listings…</div>`;
  try {
    const r = await fetch(`/api/q1?${u}`);
    const j = await r.json();
    if (!r.ok) throw new Error(j.error || "request failed");
    renderQ1(j);
  } catch (e) {
    o.innerHTML = `<div class="error">${esc(e.message)}</div>`;
  }
}

function renderQ1(j) {
  const o = $("#q1-results");
  const d = j.dates || ["?", "?"];
  const n = j.results.length;

  if (!n) {
    o.innerHTML = `
      <div class="summary-bar">
        <span><strong>0</strong> listings available in
<strong>${esc(pretty(j.city))}</strong> for <strong>${esc(d[0])} → ${esc(d[1])}
</strong></span>
        <span class="chip">no matches</span>
      </div>
      <div class="empty">No listings have both dates open. Try a different window or
city.</div>`;
    return;
  }

  const g = j.results
    .map((x, i) => {
      const r = x.review_scores_rating != null ?
Number(x.review_scores_rating).toFixed(2) : "—";
      const a = ams(x.amenities);
      const p = x.price != null ? `$${x.price}` : "—";
      const v = (i % 6) + 1;
      const ch = (x.name || "?").trim().charAt(0).toUpperCase();
      return `
        <article class="card">
          <div class="card-thumb var-${v}">${esc(ch)}</div>
          <div class="card-title">${esc(x.name || "Untitled")}</div>
          <div class="card-meta">
            <div class="row">
              <span>${esc(x.neighborhood || "—")}</span>
              <span class="rating">${esc(r)}</span>
            </div>
            <div class="row">
              <span>${esc(x.property_type || x.room_type || "")}</span>
```

```
              <span>${esc(p)} · sleeps ${esc(x.accommodates ?? "—")}</span>
            </div>
          </div>
          <div class="badges">
            <span class="badge badge-room">${esc(x.room_type || "Room")}</span>
            ${a.map((y) => `<span class="badge">${esc(y)}</span>`).join("")}
          </div>
        </article>`;
    })
    .join("");

  o.innerHTML = `
    <div class="summary-bar">
      <span><strong>${n}</strong> listings available in
<strong>${esc(pretty(j.city))}</strong> for <strong>${esc(d[0])} → ${esc(d[1])}
</strong></span>
      <span class="chip">sorted by review_scores_rating ↓</span>
    </div>
    <div class="grid">${g}</div>`;
}

// q5: december reviews per (city, year) -- expensive collection scan, show a
spinner up front
async function runQ5() {
  W.q5 = true;
  const o = $("#q5-results");
  o.innerHTML = `<div class="loading">Aggregating December reviews across all
cities…</div>`;
  try {
    const r = await fetch("/api/q5");
    const j = await r.json();
    if (!r.ok) throw new Error(j.error || "request failed");
    renderQ5(j);
  } catch (e) {
    o.innerHTML = `<div class="error">${esc(e.message)}</div>`;
  }
}

function renderQ5(j) {
  const o = $("#q5-results");
  const a = j.results || [];
  if (!a.length) {
    o.innerHTML = `<div class="empty">No December reviews in the loaded subset.
</div>`;
    return;
  }

  const g = {};
  let t = 0;
  for (const x of a) {
```

```
    if (!g[x.city]) g[x.city] = [];
    g[x.city].push(x);
    t += x.review_count;
  }

  const m = Math.max(...a.map((x) => x.review_count));

  const blocks = Object.keys(g)
    .sort()
    .map((k) => {
      const rows = g[k]
        .sort((p, q) => p.year.localeCompare(q.year))
        .map((x) => {
          const w = Math.max(2, Math.round((x.review_count / m) * 100));
          return `<div class="bar"><span class="bar-year">${esc(x.year)}</span><div
class="bar-track"><div class="bar-fill" style="width:${w}%"></div></div><span
class="bar-count">${F(x.review_count)}</span></div>`;
        })
        .join("");
      const sum = g[k].reduce((acc, x) => acc + x.review_count, 0);
      return `
        <div class="city">
          <div class="city-name">${esc(pretty(k))}</div>
          <div class="city-total">${F(sum)} December reviews · ${g[k].length}
years</div>
          ${rows}
        </div>`;
    })
    .join("");

  o.innerHTML = `
    <div class="summary-bar">
      <span><strong>${F(t)}</strong> December reviews across
<strong>${Object.keys(g).length}</strong> cities and <strong>${a.length}</strong>
(city, year) groups</span>
      <span class="chip">full-collection aggregation</span>
    </div>
    <div class="cities">${blocks}</div>`;
}

// q6: re-book reminders -- one card per repeat-reviewer/listing pair, with same-
host listings sidecar
async function runQ6() {
  W.q6 = true;
  const l = $("#q6-limit").value || 12;
  const o = $("#q6-results");
  o.innerHTML = `<div class="loading">Finding repeat reviewers and pulling same-host
listings…</div>`;
  try {
    const r = await fetch(`/api/q6?limit=${l}`);
```

```
    const j = await r.json();
    if (!r.ok) throw new Error(j.error || "request failed");
    renderQ6(j);
  } catch (e) {
    o.innerHTML = `<div class="error">${esc(e.message)}</div>`;
  }
}

// description fields can contain raw html (insideairbnb's listing pages), nuke tags
for safe display
function strip(h) {
  return String(h || "")
    .replace(/<br\s*\/?>/gi, " ")
    .replace(/<[^>]+>/g, "")
    .trim();
}

function renderQ6(j) {
  const o = $("#q6-results");
  const a = j.results || [];
  if (!a.length) {
    o.innerHTML = `<div class="empty">No repeat reviewers in the loaded subset.
</div>`;
    return;
  }

  const html = a
    .map((x) => {
      const desc = strip(x.description).slice(0, 220);
      const others = (x.other_host_listings_same_city || [])
        .slice(0, 4)
        .map((y) => `<li>${y.listing_url ? `<a href="${esc(y.listing_url)}"
target="_blank" rel="noopener">${esc(y.name || "Untitled")}</a>` : esc(y.name ||
"Untitled")}</li>`)
        .join("");
      const av = x.available_in_same_month ? `<span class="pill ok">open in
${esc(x.month)}</span>` : `<span class="pill no">no openings in ${esc(x.month)}
</span>`;
      const nights =
        x.min_nights != null || x.max_nights != null
          ? `<span class="pill">${F(x.min_nights)}–${F(x.max_nights)} nights</span>`
          : "";
      return `
        <article class="reminder">
          <div>
            <div class="reminder-head">
              <span class="reminder-month">${esc(x.month)}</span>
              <span class="reminder-host">guest <strong>#${esc(x.reviewer_id)}
</strong> · host <strong>${esc(x.host_name || "–")}</strong></span>
            </div>
```

```
            <div class="reminder-listing">${x.listing_url ? `<a
href="${esc(x.listing_url)}" target="_blank" rel="noopener">${esc(x.listing_name)}
</a>` : esc(x.listing_name)}</div>
            ${desc ? `<div class="reminder-desc">${esc(desc)}…</div>` : ""}
            <div class="reminder-meta">${av}${nights}</div>
          </div>
          <div class="reminder-side">
            <div class="label">Other listings from ${esc(x.host_name || "this
host")}</div>
            ${others ? `<ul>${others}</ul>` : `<div style="font-
size:13px;color:var(--ink-soft);">No other listings in the same city.</div>`}
          </div>
        </article>`;
    })
    .join("");

  o.innerHTML = `
    <div class="summary-bar">
      <span><strong>${a.length}</strong> repeat-reviewer reminders enriched with
calendar &amp; same-host listings</span>
      <span class="chip">1 aggregation + 4 indexed reads per record</span>
    </div>
    <div class="reminders">${html}</div>`;
}

// q2: empty neighborhoods -- shows the "dark" set as badges plus a one-line summary
bar
async function runQ2() {
  W.q2 = true;
  const c = $("#q2-city").value;
  const m = $("#q2-month").value;
  const u = new URLSearchParams({ city: c });
  if (m) u.set("month", m);
  const o = $("#q2-results");
  o.innerHTML = `<div class="loading">Building active-neighborhood set and anti-
joining…</div>`;
  try {
    const r = await fetch(`/api/q2?${u}`);
    const j = await r.json();
    if (!r.ok) throw new Error(j.error || "request failed");
    renderQ2(j);
  } catch (e) {
    o.innerHTML = `<div class="error">${esc(e.message)}</div>`;
  }
}

function renderQ2(j) {
  const o = $("#q2-results");
  const a = j.empty_neighborhoods || [];
  const t = j.total_neighborhoods || 0;
```

```
  const ac = j.active_neighborhoods || 0;
  if (!t) {
    o.innerHTML = `<div class="empty">No neighborhoods loaded for
${esc(pretty(j.city))}.</div>`;
    return;
  }
  const tiles = a.length
    ? a.map((n) => `<span class="badge badge-room">${esc(n)}</span>`).join("")
    : `<div class="empty">Every neighborhood had at least one available night in
${esc(j.month)}.</div>`;
  o.innerHTML = `
    <div class="summary-bar">
      <span><strong>${F(a.length)}</strong> of <strong>${F(t)}</strong>
${esc(pretty(j.city))} neighborhoods had no listings available in
<strong>${esc(j.month)}</strong></span>
      <span class="chip">${F(ac)} active · ${F(a.length)} dark</span>
    </div>
    <div class="badges" style="margin-top:14px">${tiles}</div>`;
}

// q3: salem entire-home bookable windows -- one card per listing, intervals listed
inside
async function runQ3() {
  W.q3 = true;
  const c = $("#q3-city").value;
  const m = $("#q3-month").value;
  const u = new URLSearchParams({ city: c });
  if (m) u.set("month", m);
  const o = $("#q3-results");
  o.innerHTML = `<div class="loading">Scanning entire-home calendars and detecting
bookable runs…</div>`;
  try {
    const r = await fetch(`/api/q3?${u}`);
    const j = await r.json();
    if (!r.ok) throw new Error(j.error || "request failed");
    renderQ3(j);
  } catch (e) {
    o.innerHTML = `<div class="error">${esc(e.message)}</div>`;
  }
}

// inclusive night count for an interval (matches the totalNights helper in q4.js)
function nightsBetween(a, b) {
  const x = new Date(`${a}T00:00:00Z`).getTime();
  const y = new Date(`${b}T00:00:00Z`).getTime();
  return Math.floor((y - x) / 86400000) + 1;
}

function renderQ3(j) {
  const o = $("#q3-results");
```

```
  const a = j.results || [];
  if (!a.length) {
    o.innerHTML = `<div class="empty">No bookable windows found for
${esc(pretty(j.city))} entire-home listings in ${esc(j.month)}.</div>`;
    return;
  }
  const limited = a.slice(0, 30);
  const totalNights = a.reduce(
    (s, x) => s + (x.intervals || []).reduce((a2, iv) => a2 +
nightsBetween(iv.from, iv.to), 0),
    0
  );
  const cards = limited
    .map((x, i) => {
      const v = (i % 6) + 1;
      const ch = (x.listing_name || "?").trim().charAt(0).toUpperCase();
      const ivs = (x.intervals || [])
        .map((iv) => `<li><strong>${esc(iv.from)}</strong> &rarr;
<strong>${esc(iv.to)}</strong> · ${nightsBetween(iv.from, iv.to)} night(s) · min
${esc(iv.min_nights)}</li>`)
        .join("");
      return `
        <article class="card">
          <div class="card-thumb var-${v}">${esc(ch)}</div>
          <div class="card-title">${esc(x.listing_name || "Untitled")}</div>
          <div class="card-meta">
            <div class="row"><span>${x.intervals.length} window(s)</span><span
class="rating">listing #${esc(x.listing_id)}</span></div>
          </div>
          <ul style="margin:10px 0 0;padding-left:18px;font-size:13px;color:var(--
ink-soft);line-height:1.55">${ivs}</ul>
        </article>`;
    })
    .join("");
  o.innerHTML = `
    <div class="summary-bar">
      <span><strong>${F(a.length)}</strong> entire-home listings have bookable
windows in <strong>${esc(j.month)}</strong> · <strong>${F(totalNights)}</strong>
total bookable nights</span>
      <span class="chip">interval scan · min-nights aware</span>
    </div>
    <div class="grid">${cards}</div>
    ${a.length > limited.length ? `<div class="hint" style="margin-top:14px">Showing
first ${limited.length} of ${F(a.length)} listings.</div>` : ""}`;
}

// q4: portland mar-aug bookable-nights trend, rendered as horizontal bar chart per
month
async function runQ4() {
  W.q4 = true;
```

```
  const o = $("#q4-results");
  o.innerHTML = `<div class="loading">Looping March–August, summing bookable nights
per month…</div>`;
  try {
    const r = await fetch("/api/q4");
    const j = await r.json();
    if (!r.ok) throw new Error(j.error || "request failed");
    renderQ4(j);
  } catch (e) {
    o.innerHTML = `<div class="error">${esc(e.message)}</div>`;
  }
}

function renderQ4(j) {
  const o = $("#q4-results");
  const m = j.months || [];
  if (!m.length) {
    o.innerHTML = `<div class="empty">No data returned.</div>`;
    return;
  }
  const max = Math.max(1, ...m.map((x) => x.total_bookable_nights));
  const total = m.reduce((s, x) => s + x.total_bookable_nights, 0);
  const rows = m
    .map((x) => {
      const w = Math.max(2, Math.round((x.total_bookable_nights / max) * 100));
      return `<div class="bar"><span class="bar-year">${esc(x.month)}</span><div
class="bar-track"><div class="bar-fill" style="width:${w}%"></div></div><span
class="bar-count">${F(x.total_bookable_nights)} nights ·
${F(x.listings_with_intervals)} listings</span></div>`;
    })
    .join("");
  o.innerHTML = `
    <div class="summary-bar">
      <span><strong>${F(total)}</strong> total bookable nights across
<strong>${esc(pretty(j.city))}</strong> entire-home listings ·
<strong>${esc(j.year)}</strong></span>
      <span class="chip">Mar &rarr; Aug</span>
    </div>
    <div class="cities"><div class="city"><div class="city-
name">${esc(pretty(j.city))} · ${esc(j.room_type)}</div>${rows}</div></div>`;
}

poll();
```